

Presentation Guide: Frontend, Middleware & Architecture Overview

Project Name: Encrypted Search via Searchable Symmetric Encryption (SSE Vault)

Presenter: Frontend & Integration Engineer

Target Audience: Technical Team, Project Managers & Stakeholders

1. Executive Summary & Core Mission

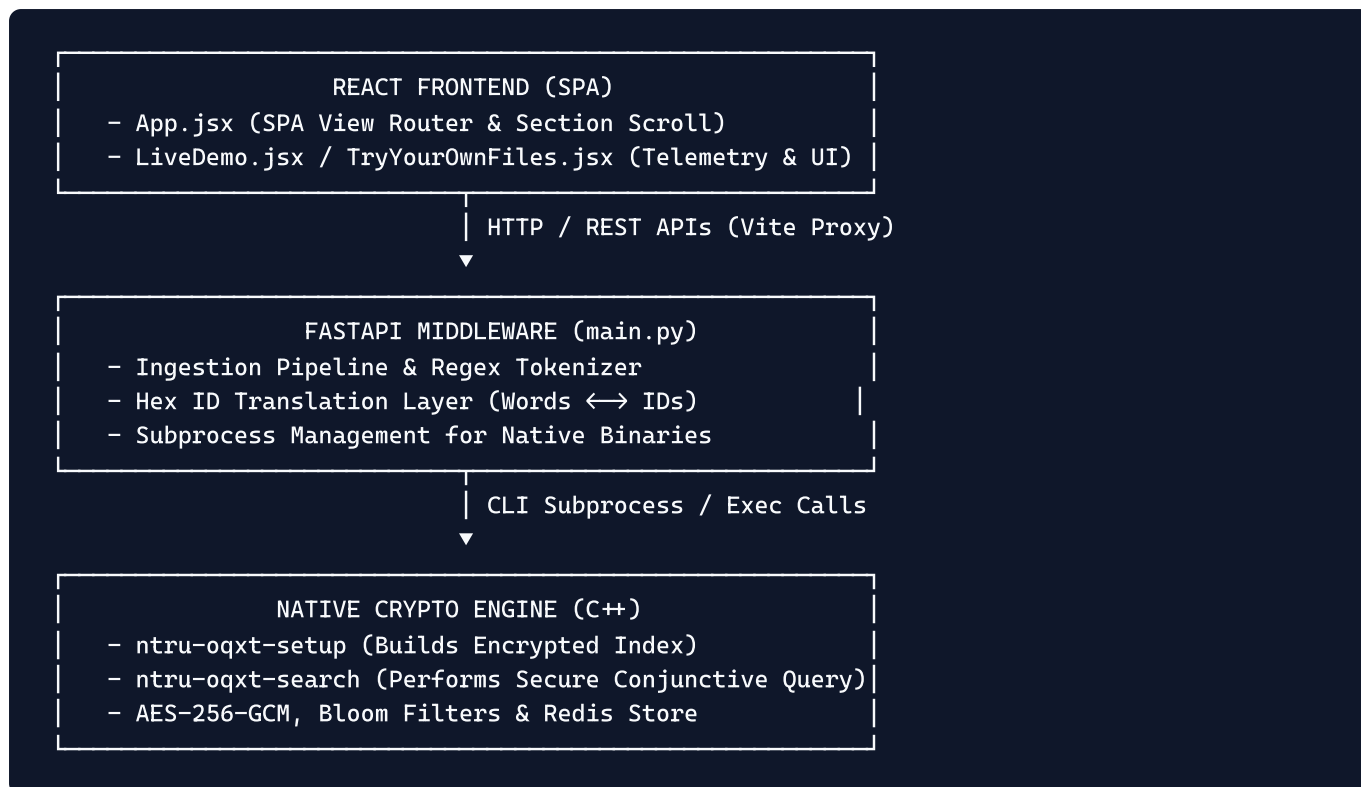
This project is a high-performance **Encrypted Search Engine** leveraging **Searchable Symmetric Encryption (SSE)** (specifically **NTRU-OQXT** and **ODXT** cryptographic protocols).

The Problem: Cloud providers and databases store user data in plaintext or encrypted at rest, but searching requires decrypting the entire database on the server, exposing sensitive data to insider threats or cloud breaches.

Our Solution: A zero-trust searchable symmetric encryption architecture. The server searches over encrypted ciphertexts and encrypted index tokens **without ever decrypting data or learning the plaintext keywords or document contents**.

Your Role (Frontend & Integration): Developed the interactive React frontend, built the SPA routing framework, integrated live telemetry UI dashboards, designed file ingestion pipelines, and connected the client seamlessly to the FastAPI middleware.

2. System Architecture: 3-Tier Model



3. Frontend Architecture & Component Breakdown

Built With:

Framework: React 18 + Vite

Styling: Custom Vanilla CSS with modern glassmorphism, responsive grid system, CSS variables (`index.css`), and micro-animations.

Iconography & UI Components: `lucide-react` icons, custom animated SVG canvas for workflow visualization (`ParticleNetwork.jsx` , `AnimatedWorkflow.jsx`).

Component Structure:

App.jsx: Main container & client-side router. Controls top-level navigation (`'home'` view vs. `'demo'` view), active section tracking (`hero` , `how` , `about` , `contact`), and smooth scrolling.

Hero.jsx: Landing page showcase with interactive CTA buttons.

LiveDemo.jsx & TryYourOwnFiles.jsx: Interactive laboratory modules where users drag & drop files, trigger setup encryption pipelines, query encrypted indices, and view live execution logs.

HowItWorks.jsx & SSEExplanation.jsx: Technical visual explanations of Searchable Symmetric Encryption.

4. Frontend Routing & Development Proxy

Client-Side SPA Routing (`App.jsx`)

Instead of using heavy full-page reloads, routing is handled dynamically in state:

```
const [page, setPage] = useState('home'); // Switches between 'home' and 'demo'

// Smooth scrolling between sections on home page
const scrollTo = (id) => {
  const el = document.getElementById(id);
  if (el) window.scrollTo({ top: el.offsetTop - 64, behavior: 'smooth' });
};
```

Vite Development Proxy (`vite.config.js`)

To avoid CORS (Cross-Origin Resource Sharing) restrictions and separate frontend dev (`localhost:3000` / `localhost:5173`) from backend API (`localhost:8000`), Vite reverse-proxies API calls:

```
// vite.config.js
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/upload': 'http://localhost:8000',
      '/stats': 'http://localhost:8000',
      '/search': 'http://localhost:8000',
      '/conjunctive-search': 'http://localhost:8000',
      '/download': 'http://localhost:8000',
      '/reset': 'http://localhost:8000',
    }
  }
})
```

Why it matters: The React frontend can make clean relative HTTP calls like `fetch('/upload')` without hardcoding backend IP addresses or tripping CORS security checks.

5. How Frontend Connects with Middleware (API Specification)

The frontend communicates with the FastAPI middleware ([main.py](#)) via REST API routes:

Route	HTTP Method	Frontend Caller	Purpose
<code>/stats</code>	GET	<code>TryYourOwnFiles.jsx</code>	Fetches real-time database KPIs (Total Docs, Vocabulary Size, Index Entries, Sample Terms).
<code>/upload</code>	POST	<code>TryYourOwnFiles.jsx</code> , <code>LiveDemo.jsx</code>	Uploads raw documents (<code>FormData</code>). Triggers tokenization, hex ID assignment, index creation, and native <code>./ntru-oqxt-setup</code> .
<code>/search?q={word}</code>	GET	<code>TryYourOwnFiles.jsx</code>	Query Resolution: Converts user plaintext search terms into 8-character Hex Word IDs (<code>0000000B</code>).
<code>/conjunctive-search</code>	POST	<code>TryYourOwnFiles.jsx</code>	Secure Execution: Sends array of Hex IDs to invoke <code>./ntru-oqxt-search</code> binary in blind cryptographic search mode.
<code>/download/{file}</code>	GET	Telemetry UI	Downloads generated lookup CSVs (<code>word_to_id.csv</code> , <code>doc_to_id.csv</code>).
<code>/reset</code>	DELETE	<code>TryYourOwnFiles.jsx</code>	Cleans up index files, purges stale binary artifacts, resets Redis state, and re-initializes database.

6. End-to-End Execution Flow (Step-by-Step Data Journey)

Phase 1: Ingestion & Encryption Flow

User Action: User drops files (`.txt`, `.pdf`, `.csv`) into the dropzone or selects sample documents.

Frontend Progress State: UI sets `isUploading = true` and updates progress indicators (Reading → Indexing → Encrypting).

HTTP POST Request: Sends `FormData` payload containing files to `/upload`.

Middleware Tokenization: `main.py` extracts text, tokenizes into lower-case words, and generates unique 8-hex-digit IDs (`00000001`, `0000000A`).

Disk & Dat Files: Saves mapping CSVs in `data/` directory and writes raw index format to `db6k.dat`.

C++ Binary Execution: `main.py` runs `./ntru-oqxt-setup` via `subprocess.run()`, constructing encrypted index tables and Bloom filters stored in Redis.

Frontend Update: Frontend receives JSON confirmation, updates KPI metric cards, and outputs a detailed execution log.

Phase 2: Secure Search Query Flow (The Cryptographic Boundary)

```
User types "medical diagnosis"
  |
  v
1. GET /search?q=medical    → Returns Word ID: 00000004
2. GET /search?q=diagnosis → Returns Word ID: 0000000C
  |
  v
3. POST /conjunctive-search { word_ids: ["00000004", "0000000C"] }
  |
  v
Middleware runs: ./ntru-oqxt-search 00000004 0000000C
  |
  v
Native C++ Engine queries encrypted Redis structures (No plaintext exposed)
  |
  v
Returns matching Document IDs: ["00000002", "00000005"]
  |
  v
Middleware maps back to ["Medical_Records.pdf", "Clinical_Audit.txt"]
  |
  v
Frontend renders matching file result cards & execution timing (e.g. 0.28ms)!
```

7. Slide-by-Slide Presentation Structure for Tomorrow

Here is your exact slide breakdown to present to your team:

Slide 1: Title & Overview

Title: SSE Vault — Frontend Architecture & Middleware Integration

Key Point: Delivering a seamless, intuitive React UI over a high-performance C++ Searchable Symmetric Encryption engine.

Slide 2: 3-Tier Architectural Model

Explain the separation of concerns: React SPA (UI) $\rightarrow$ FastAPI Middleware (Tokenization & ID mapping) $\rightarrow$ Native C++ Engine (SSE Crypto).

Highlight why separation is key for security: The cryptographic C++ layer never handles raw text.

Slide 3: Frontend Component Design & SPA Routing

Demo the SPA structure (`App.jsx`), state-driven view switching (`home` vs `demo`), and sticky navigation bar.

Explain the user experience design: Rich glassmorphism aesthetics, drag-and-drop file ingestion, live telemetry logging console, and progress bars.

Slide 4: Vite Proxy & Networking Setup

Explain how `vite.config.js` routes traffic seamlessly between port 5173/3000 and the FastAPI backend on port 8000.

Mention developer productivity: Zero CORS issues, clean relative endpoints (`/upload` , `/conjunctive-search`).

Slide 5: Middleware Endpoints & Hex ID Translation

Detail the core REST API endpoints (`/upload` , `/stats` , `/search` , `/conjunctive-search` , `/reset`).

Explain the **Hex ID Abstraction Layer**: Why and how plaintext strings are converted into 8-character hex tokens before cryptographic operations.

Slide 6: Step-by-Step Live Demo Walkthrough

Walk through uploading documents $\rightarrow$ indexing $\rightarrow$ running a secure conjunctive query.

Point out key UI metrics: Vocabulary size, search execution latency ($< 1\text{ms}$), and matching document list.

8. Potential Presentation Questions & Answers (Q&A Prep)

Q1: Why did you use FastAPI as the middleware instead of calling C++ directly from Node/React?

Answer: FastAPI provides high-performance asynchronous Python request handling with built-in Pydantic validation, easy file streaming (`UploadFile`), and straightforward C++ binary orchestration using Python's `subprocess` module.

Q2: How does the frontend handle search for words that don't exist in the uploaded files?

Answer: During the query resolution phase (`GET /search?q=<word>`), the frontend checks if the keyword exists in the dictionary. If any term is missing, the search aborts early on the client with a clear notification, avoiding unnecessary cryptographic binary calls.

Q3: How does the frontend stay responsive during long upload/encryption operations?

Answer: All upload and setup requests use asynchronous `fetch` promises in React. We track step state (`uploadStep`: Reading $\rightarrow$ Indexing $\rightarrow$ Encrypting) and display an interactive progress bar so the main UI thread is never blocked.

Q4: How does Vite proxy work in production vs development?

Answer: In development, Vite acts as a local proxy server forwarding `/upload` requests to port 8000. In production, Docker Compose uses Nginx to serve built static assets and proxy `/` endpoints to the FastAPI service container directly.